

DA01 – DATA ANALYTICS PROJECT

Predicting Late Deliveries in Brazilian E-Commerce

University of Europe for Applied Sciences | May 2026

Authors: Renato Silva | Sandra Raj Pattuvakkaran | Sudarsh Mekampurath Sajeev

How to run:

1. Upload all 9 `*_clean.csv` files from the DE pipeline when prompted
2. Go to **Runtime** → **Run all**
3. All plots and CSVs save automatically

Cell 1 — Install Dependencies

```
!pip install imbalanced-learn xgboost --quiet
print('✅ Dependencies installed.')
```

✅ Dependencies installed.

Cell 2 — Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
np.random.seed(42) # global seed for full reproducibility

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve, f1_score,
                             ConfusionMatrixDisplay)

from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE
from scipy import stats
from itertools import combinations
import sklearn, xgboost, imblearn

print('✅ All libraries imported successfully.')
print(f' NumPy: {np.__version__} | Pandas: {pd.__version__}')
print(f' Scikit-learn: {sklearn.__version__} | XGBoost: {xgboost.__version__}')
print(f' imbalanced-learn: {imblearn.__version__}')
```

✅ All libraries imported successfully.
NumPy: 2.0.2 | Pandas: 2.2.2
Scikit-learn: 1.6.1 | XGBoost: 3.3.0
imbalanced-learn: 0.14.2

Cell 3 — Upload Cleaned CSV Files (from DE Pipeline)

```
from google.colab import files
uploaded = files.upload()
# Upload all 9 *_clean.csv files when prompted:
# orders_clean.csv, order_items_clean.csv, payments_clean.csv,
# customers_clean.csv, sellers_clean.csv, products_clean.csv,
# reviews_clean.csv, geolocation_clean.csv, category_translation_clean.csv
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving order_items_clean.csv to order_items_clean.csv
 Saving geolocation_clean.csv to geolocation_clean.csv
 Saving customers_clean.csv to customers_clean.csv
 Saving category_translation_clean.csv to category_translation_clean.csv
 Saving sellers_clean.csv to sellers_clean.csv
 Saving reviews_clean.csv to reviews_clean.csv
 Saving products_clean.csv to products_clean.csv
 Saving payments_clean.csv to payments_clean.csv

Cell 4 — Load All 9 Cleaned CSV Files

```
import pandas as pd

orders      = pd.read_csv('orders_clean.csv')
order_items = pd.read_csv('order_items_clean.csv')
payments    = pd.read_csv('payments_clean.csv')
customers    = pd.read_csv('customers_clean.csv')
sellers      = pd.read_csv('sellers_clean.csv')
products     = pd.read_csv('products_clean.csv')
reviews      = pd.read_csv('reviews_clean.csv')
geolocation  = pd.read_csv('geolocation_clean.csv')
cat_trans    = pd.read_csv('category_translation_clean.csv')

print(f'Orders:      {orders.shape}')
print(f'Order Items: {order_items.shape}')
print(f'Payments:     {payments.shape}')
print(f'Customers:    {customers.shape}')
print(f'Sellers:      {sellers.shape}')
print(f'Products:     {products.shape}')
print(f'Reviews:      {reviews.shape}')
print(f'Geolocation:   {geolocation.shape}')
print(f'Categories:    {cat_trans.shape}')
print('✅ All datasets loaded successfully.')
```

```
Orders:      (99441, 8)
Order Items: (112650, 7)
Payments:    (103886, 5)
Customers:   (99441, 5)
Sellers:     (3095, 4)
Products:    (32951, 9)
Reviews:     (99224, 7)
Geolocation: (1000163, 5)
Categories:  (71, 2)
✅ All datasets loaded successfully.
```

Cell 5 — Parse Datetime Columns

```
date_cols = [
    'order_purchase_timestamp',
    'order_approved_at',
    'order_delivered_carrier_date',
    'order_delivered_customer_date',
    'order_estimated_delivery_date'
]
for col in date_cols:
    orders[col] = pd.to_datetime(orders[col], errors='coerce')

print('✅ Datetime columns parsed.')
print(f'   Date range: {orders["order_purchase_timestamp"].min().date()} → '
      f'{orders["order_purchase_timestamp"].max().date()}')
```

```
✅ Datetime columns parsed.
Date range: 2016-09-04 → 2018-10-17
```

Cell 6 — Build orders_enriched: Feature Engineering & Target Variable

```

# Keep only delivered orders
df = orders[orders['order_delivered_customer_date'].notna()].copy()

# --- Target variable & delivery KPIs ---
df['delivery_days'] = (
    df['order_delivered_customer_date'] - df['order_purchase_timestamp']
).dt.total_seconds() / 86400

df['days_late'] = (
    df['order_delivered_customer_date'] - df['order_estimated_delivery_date']
).dt.total_seconds() / 86400

df['is_late'] = (df['days_late'] > 0).astype(int)

# --- Time-based features (available at order time) ---
df['order_hour'] = df['order_purchase_timestamp'].dt.hour
df['order_day_of_week'] = df['order_purchase_timestamp'].dt.dayofweek # 0=Monday

# --- Order items aggregates ---
items_agg = order_items.groupby('order_id').agg(
    item_count=('order_item_id', 'count'),
    price=('price', 'sum'),
    freight_value=('freight_value', 'sum')
).reset_index()
df = df.merge(items_agg, on='order_id', how='left')

# --- Payments aggregates ---
pay_agg = payments.groupby('order_id').agg(
    payment_installments=('payment_installments', 'max'),
    payment_value=('payment_value', 'sum'),
    payment_type=('payment_type', lambda x: x.mode()[0] if len(x) > 0 else 'unknown')
).reset_index()
df = df.merge(pay_agg, on='order_id', how='left')

# --- Revenue per order ---
df['revenue_per_order'] = df['price'] + df['freight_value']

# --- Product dimensions (mean per order) ---
items_products = order_items.merge(products, on='product_id', how='left')
prod_agg = items_products.groupby('order_id').agg(
    product_weight_g=('product_weight_g', 'mean'),
    product_length_cm=('product_length_cm', 'mean'),
    product_height_cm=('product_height_cm', 'mean'),
    product_width_cm=('product_width_cm', 'mean')
).reset_index()
df = df.merge(prod_agg, on='order_id', how='left')

# --- Customer state ---
df = df.merge(customers[['customer_id', 'customer_state']], on='customer_id', how='left')

# --- Seller state (first seller per order) ---
seller_per_order = order_items[['order_id', 'seller_id']].drop_duplicates('order_id')
seller_per_order = seller_per_order.merge(
    sellers[['seller_id', 'seller_state']], on='seller_id', how='left')
df = df.merge(seller_per_order[['order_id', 'seller_state']], on='order_id', how='left')

# --- Review score ---
rev_agg = reviews.groupby('order_id')['review_score'].mean().reset_index()
df = df.merge(rev_agg, on='order_id', how='left')

# --- Review sentiment label ---
def sentiment(score):
    if pd.isna(score): return 'no_review'
    if score >= 4: return 'positive'
    if score == 3: return 'neutral'
    return 'negative'
df['review_sentiment_label'] = df['review_score'].apply(sentiment)

# --- Engineered features ---
df['same_state'] = (df['seller_state'] == df['customer_state']).astype(int)
df['product_volume'] = df['product_length_cm'] * df['product_height_cm'] * df['product_width_cm']
df['payment_ratio'] = df['freight_value'] / df['revenue_per_order'].replace(0, np.nan)

print(f'orders_enriched shape: {df.shape}')

```

```
print(f'Late orders:      {df["is_late"].sum():,} ({df["is_late"].mean()*100:.1f}%)')
print(f'On-time orders: {(df["is_late"]==0).sum():,} ({(df["is_late"]==0).mean()*100:.1f}%)')
print(f'Mean delivery days:      {df["delivery_days"].mean():.1f}')
print(f'Mean days late (if late):{df[df["is_late"]==1]["days_late"].mean():.1f}')
print('✅ orders_enriched built.')
```

```
orders_enriched shape: (96476, 31)
Late orders:      7,827 (8.1%)
On-time orders: 88,649 (91.9%)
Mean delivery days:      12.6
Mean days late (if late):9.6
✅ orders_enriched built.
```

Cell 7 — EDA: Class Distribution

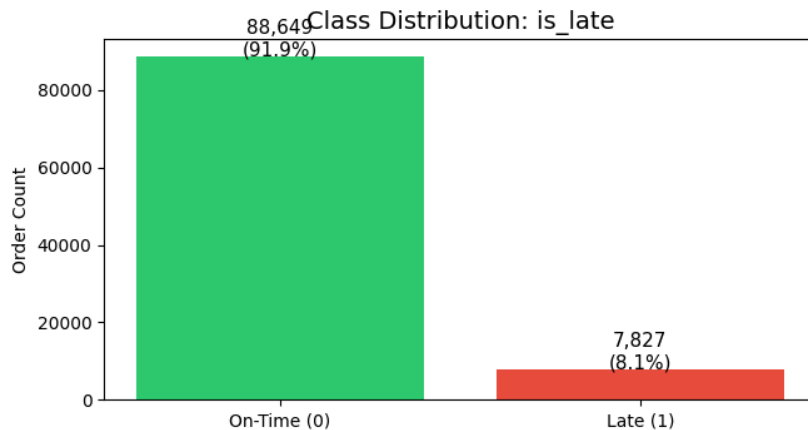
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
counts = df['is_late'].value_counts()

axes[0].bar(['On-Time (0)', 'Late (1)'], counts.values, color=['#2ecc71', '#e74c3c'])
axes[0].set_title('Class Distribution: is_late', fontsize=14)
axes[0].set_ylabel('Order Count')
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 200, f'{v:,}\n({v/len(df)*100:.1f}%', ha='center', fontsize=11)

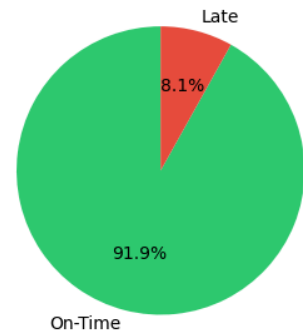
axes[1].pie(counts.values, labels=['On-Time', 'Late'], autopct='%1.1f%%',
            colors=['#2ecc71', '#e74c3c'], startangle=90)
axes[1].set_title('Late vs On-Time Split', fontsize=14)

plt.suptitle('Class Imbalance Overview', fontsize=15, y=1.02)
plt.tight_layout()
plt.savefig('eda_class_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ Class distribution plotted.')
print(f'⚠ Imbalance ratio: {(df["is_late"]==0).mean()*100:.0f}:{(df["is_late"]==1).mean()*100:.0f}')
print('Solution: SMOTE on training data only + F1/AUC as primary metrics')
```

Class Imbalance Overview



Late vs On-Time Split



```
✅ Class distribution plotted.
⚠ Imbalance ratio: 92:8
Solution: SMOTE on training data only + F1/AUC as primary metrics
```

Cell 8 — EDA: Delivery Days Histogram

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(df['delivery_days'].dropna(), bins=50, color='#3498db', edgecolor='white')
axes[0].set_title('Distribution of Delivery Days', fontsize=13)
axes[0].set_xlabel('Days from Purchase to Delivery')
axes[0].set_ylabel('Count')
axes[0].axvline(df['delivery_days'].median(), color='red', linestyle='--',
                label=f'Median: {df["delivery_days"].median():.1f} days')
axes[0].legend()
```

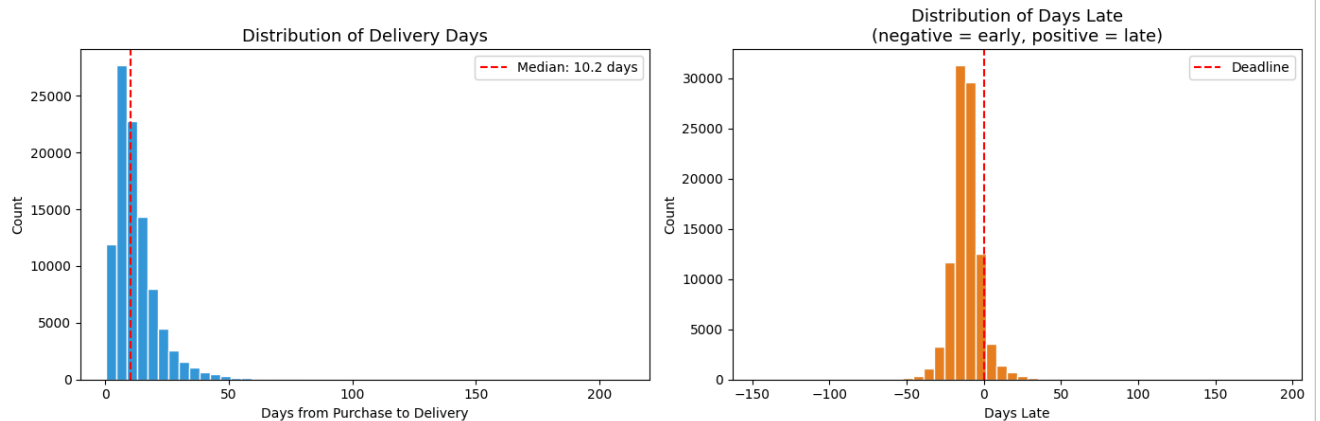
```

axes[1].hist(df['days_late'].dropna(), bins=50, color='#e67e22', edgecolor='white')
axes[1].set_title('Distribution of Days Late\n(negative = early, positive = late)', fontsize=13)
axes[1].set_xlabel('Days Late')
axes[1].set_ylabel('Count')
axes[1].axvline(0, color='red', linestyle='--', label='Deadline')
axes[1].legend()

plt.suptitle('Delivery Time Analysis', fontsize=15, y=1.02)
plt.tight_layout()
plt.savefig('eda_delivery_histogram.png', dpi=150, bbox_inches='tight')
plt.show()

```

Delivery Time Analysis



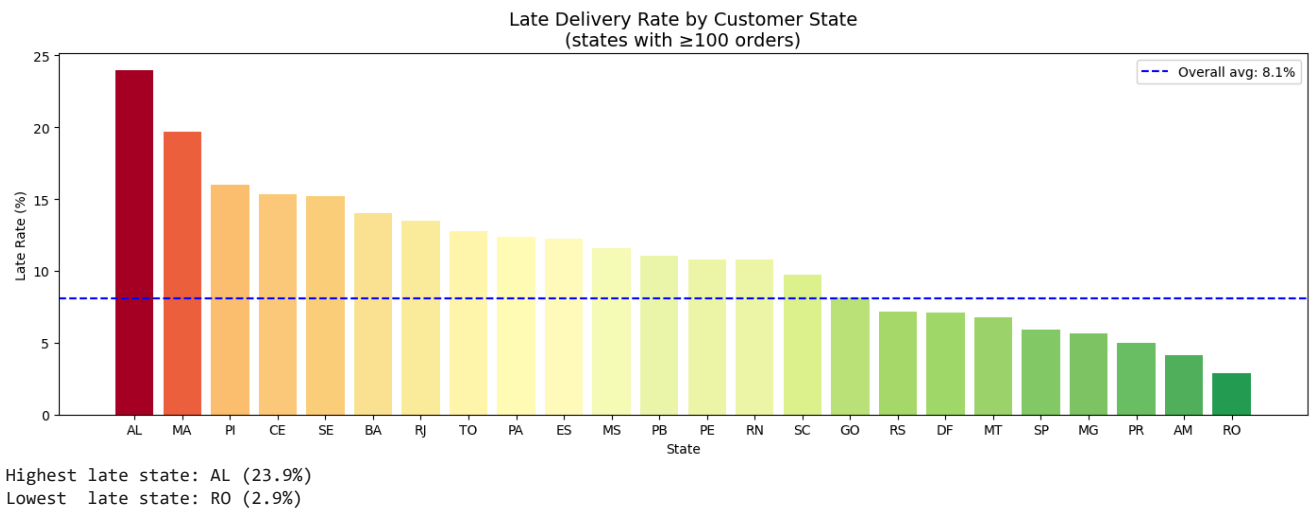
Cell 9 — EDA: Late Delivery Rate by Customer State

```

state_late = df.groupby('customer_state')['is_late'].agg(['mean', 'count']).reset_index()
state_late.columns = ['state', 'late_rate', 'order_count']
state_late = state_late[state_late['order_count'] >= 100].sort_values('late_rate', ascending=False)

plt.figure(figsize=(14, 5))
bars = plt.bar(state_late['state'], state_late['late_rate'] * 100,
               color=plt.cm.RdYlGn_r(state_late['late_rate'] / state_late['late_rate'].max()))
plt.title('Late Delivery Rate by Customer State\n(states with ≥100 orders)', fontsize=14)
plt.xlabel('State')
plt.ylabel('Late Rate (%)')
plt.axhline(df['is_late'].mean() * 100, color='blue', linestyle='--',
            label=f'Overall avg: {df["is_late"].mean()*100:.1f}%')
plt.legend()
plt.tight_layout()
plt.savefig('eda_late_by_state.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'Highest late state: {state_late.iloc[0]["state"]} ({state_late.iloc[0]["late_rate"]*100:.1f}%)')
print(f'Lowest late state: {state_late.iloc[-1]["state"]} ({state_late.iloc[-1]["late_rate"]*100:.1f}%)')

```

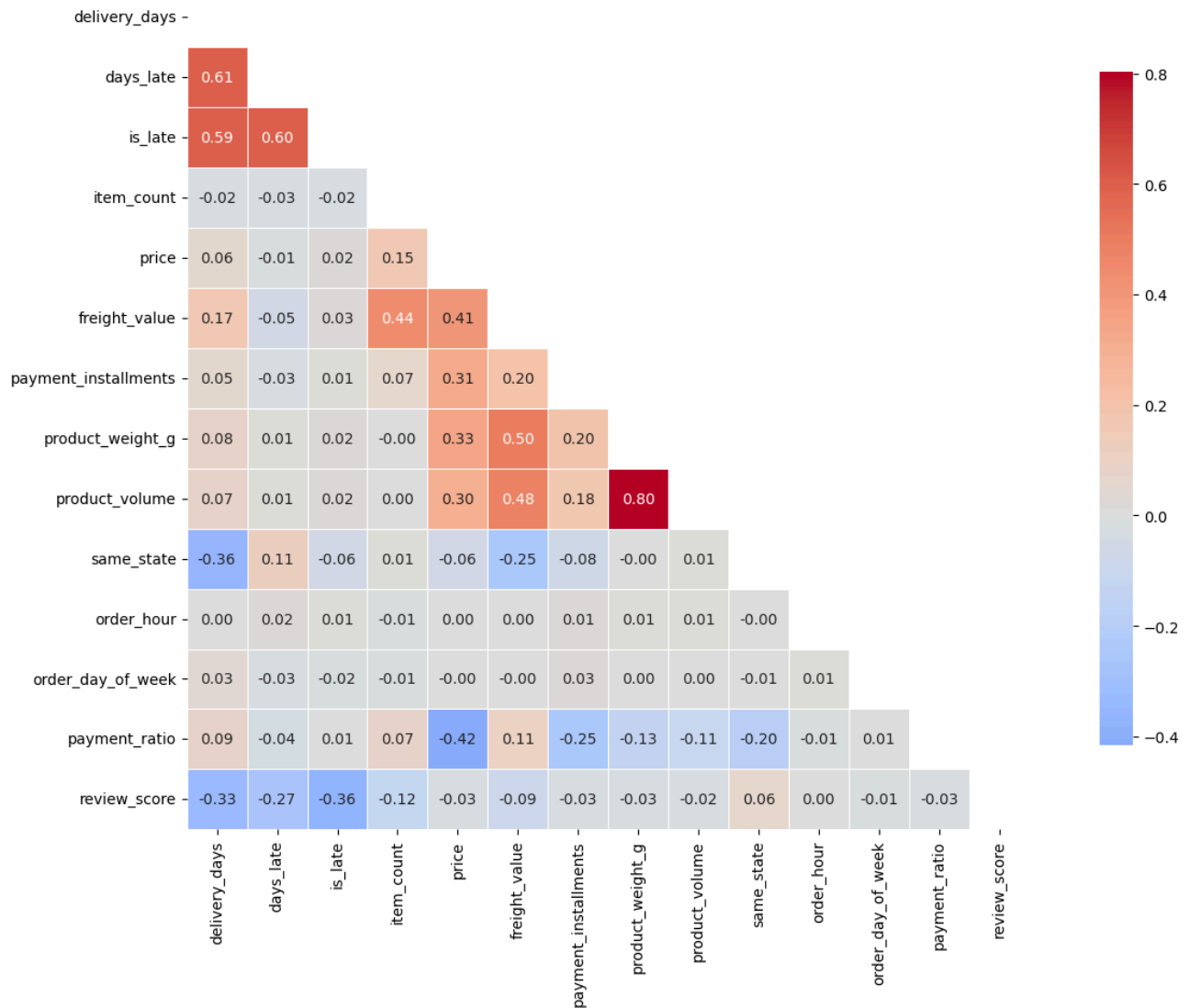


Cell 10 — EDA: Correlation Heatmap

```
num_features = [
    'delivery_days', 'days_late', 'is_late', 'item_count', 'price',
    'freight_value', 'payment_installments', 'product_weight_g',
    'product_volume', 'same_state', 'order_hour', 'order_day_of_week',
    'payment_ratio', 'review_score'
]
corr = df[num_features].corr()

plt.figure(figsize=(14, 10))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=0.5, cbar_kws={'shrink': 0.8})
plt.title('Correlation Heatmap - Numeric Features', fontsize=14)
plt.tight_layout()
plt.savefig('eda_correlation_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
print('Top correlations with is_late:')
corr_target = corr['is_late'].drop('is_late').abs().sort_values(ascending=False)
for col, val in corr_target.head(5).items():
    print(f'    {col:30s}: {val:.3f}')
```

Correlation Heatmap — Numeric Features



Top correlations with is_late:

```

days_late      : 0.605
delivery_days   : 0.590
review_score    : 0.365
same_state      : 0.056
freight_value   : 0.025

```

Cell 11 — EDA: Box Plots of Key Features vs is_late

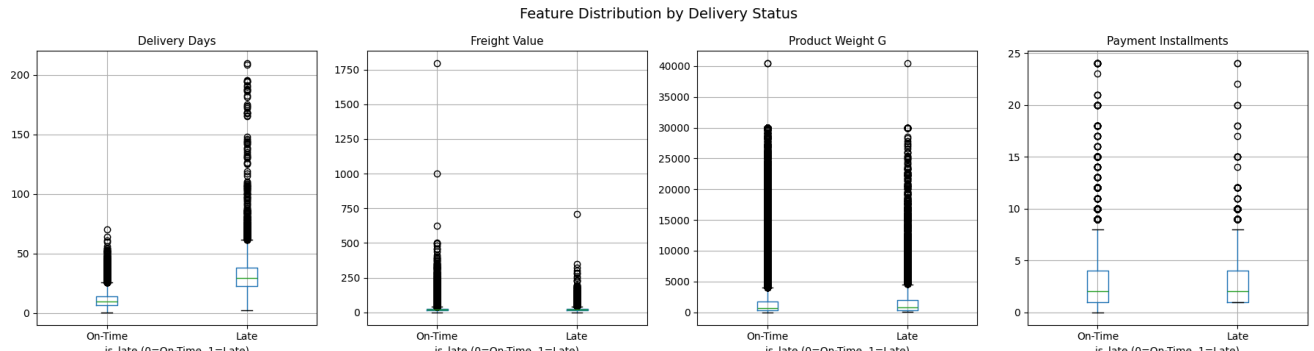
```

box_features = ['delivery_days', 'freight_value', 'product_weight_g', 'payment_installments']
fig, axes = plt.subplots(1, 4, figsize=(18, 5))

for ax, feat in zip(axes, box_features):
    df.boxplot(column=feat, by='is_late', ax=ax)
    ax.set_title(feat.replace('_', ' ').title(), fontsize=11)
    ax.set_xlabel('is_late (0=On-Time, 1=Late)')
    ax.set_xticklabels(['On-Time', 'Late'])

plt.suptitle('Feature Distribution by Delivery Status', fontsize=14)
plt.tight_layout()
plt.savefig('eda_boxplots.png', dpi=150, bbox_inches='tight')
plt.show()

```



Cell 12 — Preprocessing: Feature Selection & Label Encoding

```
feature_cols = [
    'item_count', 'price', 'freight_value', 'payment_installments',
    'product_weight_g', 'product_length_cm', 'product_height_cm',
    'product_width_cm', 'product_volume', 'same_state',
    'order_hour', 'order_day_of_week', 'payment_ratio',
    'customer_state', 'seller_state', 'payment_type'
]
target_col = 'is_late'

# Drop rows with NaN in features or target
model_df = df[feature_cols + [target_col]].dropna().copy()
print(f'Modelling dataset shape: {model_df.shape}')

# Label encode categorical columns
cat_cols = ['customer_state', 'seller_state', 'payment_type']
le = LabelEncoder()
for col in cat_cols:
    model_df[col] = le.fit_transform(model_df[col].astype(str))

X = model_df[feature_cols]
y = model_df[target_col]

print(f'Features: {X.shape[1]} | Samples: {X.shape[0]}')
print(f'Class balance - 0 (On-Time): {(y==0).sum():,} | 1 (Late): {(y==1).sum():,}')
```

```
Modelling dataset shape: (96475, 17)
Features: 16 | Samples: 96475
Class balance - 0 (On-Time): 88,649 | 1 (Late): 7,826
```

Cell 13 — Train/Test Split (Stratified 80/20)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print(f'Train: {X_train.shape} | Test: {X_test.shape}')
print(f'Train late rate: {y_train.mean()*100:.1f}% (stratified - same ratio preserved)')
print(f'Test late rate: {y_test.mean()*100:.1f}%')
```

```
Train: (77180, 16) | Test: (19295, 16)
Train late rate: 8.1% (stratified - same ratio preserved)
Test late rate: 8.1%
```

Cell 14 — StandardScaler (fit on train only — prevents data leakage)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on train only
X_test_scaled = scaler.transform(X_test)      # transform test with same params
print('✅ StandardScaler applied - fit on training data only (no data leakage).')
```

```
✅ StandardScaler applied - fit on training data only (no data leakage).
```


Cell 15 — SMOTE: Handle Class Imbalance (Training Set Only)

```
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train_scaled, y_train)

print(f'Before SMOTE - 0: {(y_train==0).sum():,} | 1: {(y_train==1).sum():,}')
print(f'After SMOTE - 0: {(y_train_sm==0).sum():,} | 1: {(y_train_sm==1).sum():,}')
print('✅ SMOTE applied to training set only - test set remains original distribution.')
```

```
Before SMOTE - 0: 70,919 | 1: 6,261
After SMOTE - 0: 70,919 | 1: 70,919
✅ SMOTE applied to training set only - test set remains original distribution.
```

Cell 16 — Define All 5 Classification Models

```
models = {
    'Logistic Regression': LogisticRegression(
        max_iter=1000, class_weight='balanced', random_state=42
    ),
    'Decision Tree': DecisionTreeClassifier(
        max_depth=6, class_weight='balanced', random_state=42
    ),
    'Random Forest': RandomForestClassifier(
        n_estimators=100, max_depth=8, class_weight='balanced',
        random_state=42, n_jobs=-1
    ),
    'KNN': KNeighborsClassifier(
        n_neighbors=11, n_jobs=-1
    ),
    'XGBoost': XGBClassifier(
        n_estimators=200, max_depth=5, learning_rate=0.05,
        eval_metric='logloss', random_state=42, n_jobs=-1,
        scale_pos_weight=(y_train==0).sum() / (y_train==1).sum()
    )
}

print('✅ All 5 models defined:')
for name in models:
    print(f'    > {name}')
```

```
✅ All 5 models defined:
> Logistic Regression
> Decision Tree
> Random Forest
> KNN
> XGBoost
```

Cell 17 — 10-Fold Cross-Validation (F1-Score)

H0: No significant difference in F1-score between models

H1: Significant difference exists (tested via paired t-test in Cell 21)

```
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
cv_results = {}

print('Running 10-Fold CV (F1-score) ...')
for name, model in models.items():
    scores = cross_val_score(model, X_train_sm, y_train_sm,
                             cv=cv, scoring='f1', n_jobs=-1)
    cv_results[name] = scores
    print(f'    {name:22s} → Mean F1: {scores.mean():.4f} Std: {scores.std():.4f}')

print('\n✅ Cross-validation complete.')
```

```
Running 10-Fold CV (F1-score) ...
Logistic Regression    → Mean F1: 0.5818 Std: 0.0050
Decision Tree          → Mean F1: 0.6723 Std: 0.0045
Random Forest          → Mean F1: 0.7329 Std: 0.0060
KNN                    → Mean F1: 0.8299 Std: 0.0018
XGBoost                → Mean F1: 0.7206 Std: 0.0016
```

```
✅ Cross-validation complete.
```

Cell 18 — Train All Models & Evaluate on Original Test Set

```

test_results = {}

print('Training & evaluating on test set ...\n')
for name, model in models.items():
    model.fit(X_train_sm, y_train_sm)
    y_pred = model.predict(X_test_scaled)
    y_proba = model.predict_proba(X_test_scaled)[:, 1]

    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_proba)
    report = classification_report(y_test, y_pred, output_dict=True)

    test_results[name] = {
        'model': model,
        'y_pred': y_pred,
        'y_proba': y_proba,
        'f1': f1,
        'auc': auc,
        'precision': report['1']['precision'],
        'recall': report['1']['recall'],
        'accuracy': report['accuracy']
    }

    print(f'{'='*50}')
    print(f' {name}')
    print(f' Accuracy: {report["accuracy"]:.4f}')
    print(f' F1-Score: {f1:.4f}')
    print(f' ROC-AUC: {auc:.4f}')
    print(f' Precision: {report["1"]["precision"]:.4f}')
    print(f' Recall: {report["1"]["recall"]:.4f}')

```

Training & evaluating on test set ...

```

=====
Logistic Regression
Accuracy: 0.5571
F1-Score: 0.1751
ROC-AUC: 0.5895
Precision: 0.1031
Recall: 0.5796
=====
Decision Tree
Accuracy: 0.5565
F1-Score: 0.1688
ROC-AUC: 0.5603
Precision: 0.0995
Recall: 0.5553
=====
Random Forest
Accuracy: 0.6811
F1-Score: 0.1693
ROC-AUC: 0.5808
Precision: 0.1073
Recall: 0.4006
=====
KNN
Accuracy: 0.6045
F1-Score: 0.1597
ROC-AUC: 0.5639
Precision: 0.0965
Recall: 0.4633
=====
XGBoost
Accuracy: 0.2887
F1-Score: 0.1650
ROC-AUC: 0.6134
Precision: 0.0912
Recall: 0.8665

```

Cell 19 — Model Comparison Table

```

results_df = pd.DataFrame({
    name: {
        'Accuracy': v['accuracy'],
        'Precision': v['precision'],

```

```

        'Recall': v['recall'],
        'F1-Score': v['f1'],
        'ROC-AUC': v['auc'],
        'CV F1 Mean': cv_results[name].mean(),
        'CV F1 Std': cv_results[name].std()
    }
    for name, v in test_results.items()
}).T.round(4)

print('\n📊 Model Comparison Table:')
print(results_df.to_string())

best_f1 = results_df['F1-Score'].idxmax()
best_auc = results_df['ROC-AUC'].idxmax()
print(f'\n🏆 Best model by F1-Score: {best_f1} ({results_df.loc[best_f1, "F1-Score"]:.4f})')
print(f'🏆 Best model by ROC-AUC: {best_auc} ({results_df.loc[best_auc, "ROC-AUC"]:.4f})')

results_df.to_csv('model_results.csv')
print('\n✅ model_results.csv saved.')

```

📊 Model Comparison Table:

	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV F1 Mean	CV F1 Std
Logistic Regression	0.5571	0.1031	0.5796	0.1751	0.5895	0.5818	0.0050
Decision Tree	0.5565	0.0995	0.5553	0.1688	0.5603	0.6723	0.0045
Random Forest	0.6811	0.1073	0.4006	0.1693	0.5808	0.7329	0.0060
KNN	0.6045	0.0965	0.4633	0.1597	0.5639	0.8299	0.0018
XGBoost	0.2887	0.0912	0.8665	0.1650	0.6134	0.7206	0.0016

🏆 Best model by F1-Score: Logistic Regression (0.1751)
 🏆 Best model by ROC-AUC: XGBoost (0.6134)

✅ model_results.csv saved.

Cell 20 — ROC Curves (All 5 Models)

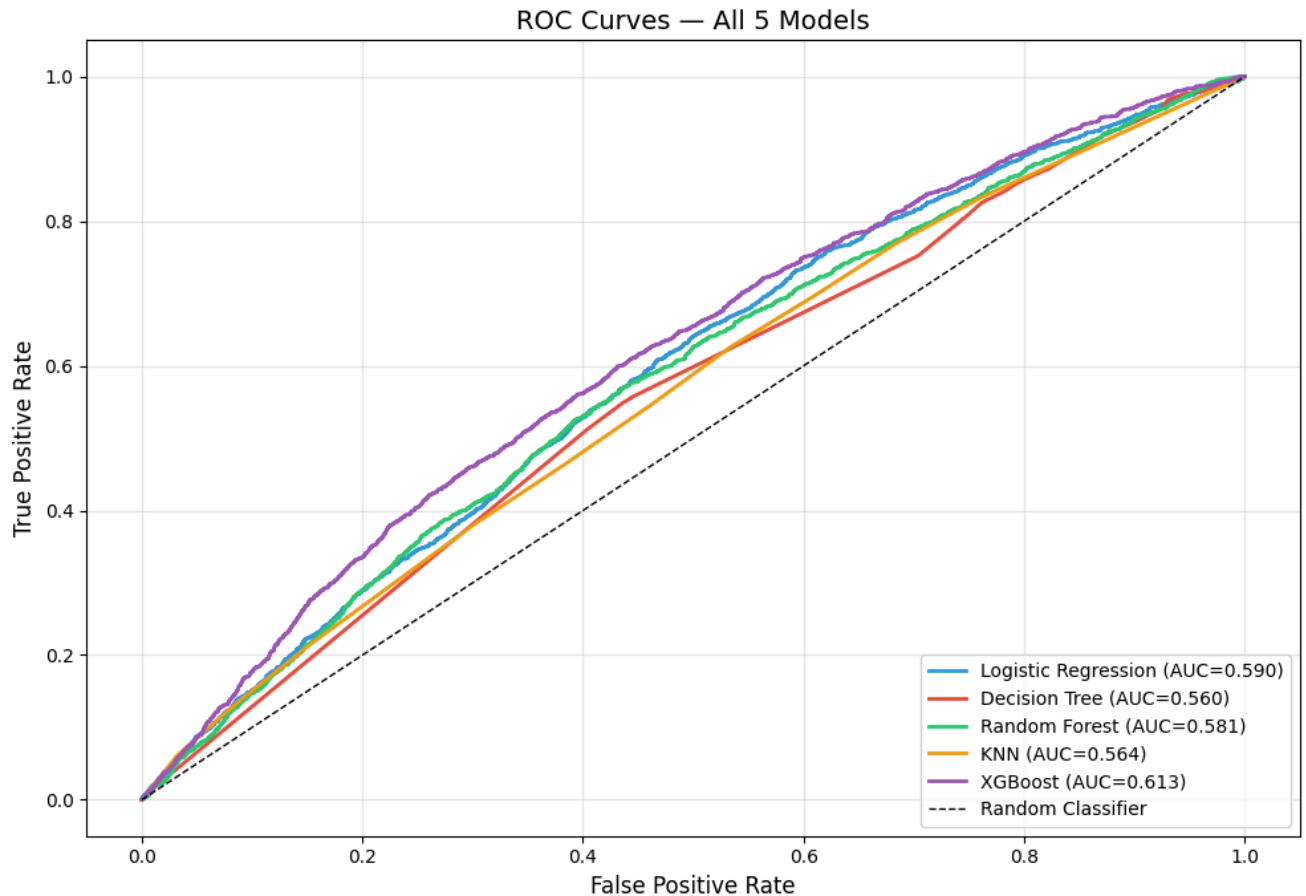
```

plt.figure(figsize=(10, 7))
colors = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12', '#9b59b6']

for (name, v), color in zip(test_results.items(), colors):
    fpr, tpr, _ = roc_curve(y_test, v['y_proba'])
    plt.plot(fpr, tpr, label=f'{name} (AUC={v["auc"]:.3f})', color=color, lw=2)

plt.plot([0, 1], [0, 1], 'k--', lw=1, label='Random Classifier')
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curves - All 5 Models', fontsize=14)
plt.legend(loc='lower right', fontsize=10)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('roc_curves.png', dpi=150, bbox_inches='tight')
plt.show()

```

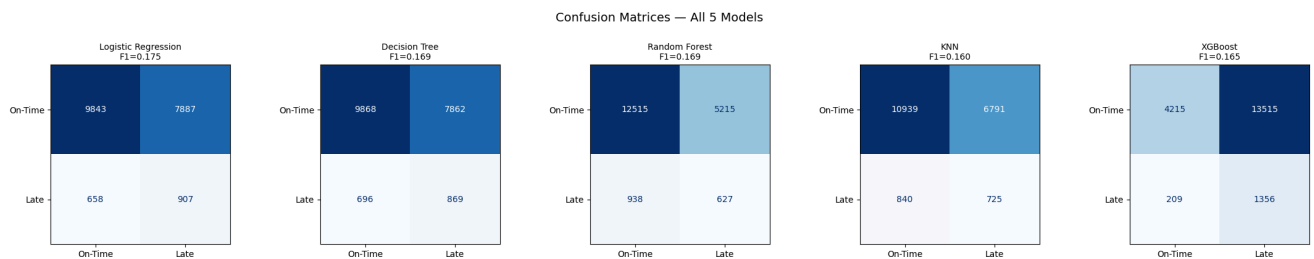


Cell 21 — Confusion Matrices (All 5 Models)

```
fig, axes = plt.subplots(1, 5, figsize=(22, 4))

for ax, (name, v) in zip(axes, test_results.items()):
    cm = confusion_matrix(y_test, v['y_pred'])
    disp = ConfusionMatrixDisplay(cm, display_labels=['On-Time', 'Late'])
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(f'{name}\nF1={v["f1"]:.3f}', fontsize=10)
    ax.set_xlabel('')
    ax.set_ylabel('')

plt.suptitle('Confusion Matrices — All 5 Models', fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig('confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
```



Cell 22 — Feature Importance (Random Forest + XGBoost)

```
fig, axes = plt.subplots(1, 2, figsize=(18, 6))

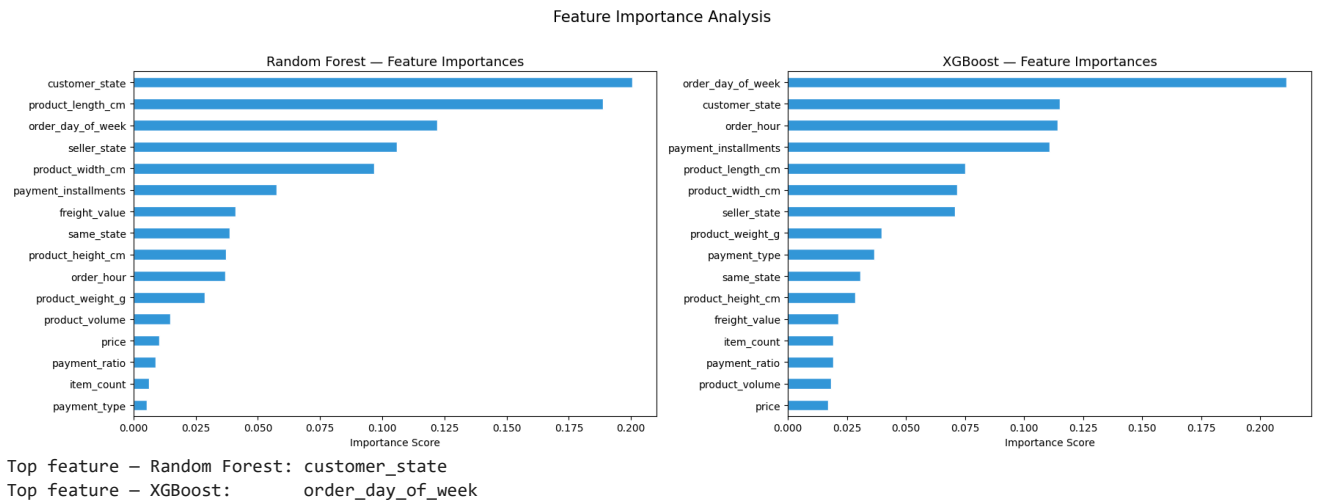
for ax, model_name in zip(axes, ['Random Forest', 'XGBoost']):
    model_obj = test_results[model_name]['model']
```

```

importances = model_obj.feature_importances_
feat_imp = pd.Series(importances, index=feature_cols).sort_values(ascending=True)
feat_imp.plot(kind='barh', ax=ax, color='#3498db', edgecolor='white')
ax.set_title(f'{model_name} - Feature Importances', fontsize=13)
ax.set_xlabel('Importance Score')

plt.suptitle('Feature Importance Analysis', fontsize=15, y=1.02)
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()
rf_top = pd.Series(test_results['Random Forest']['model'].feature_importances_, index=feature_cols).idxmax()
xgb_top = pd.Series(test_results['XGBoost']['model'].feature_importances_, index=feature_cols).idxmax()
print(f'Top feature - Random Forest: {rf_top}')
print(f'Top feature - XGBoost: {xgb_top}')

```



Cell 23 — Statistical Testing: Paired t-Test on 10-Fold CV F1 Scores

H0: No significant difference in F1-score between two models

H1: A significant difference exists

Significance level: $p < 0.05$

```

model_names = list(cv_results.keys())
print('📊 Paired t-test on 10-Fold CV F1 Scores (p < 0.05 = significant)\n')
print(f'{"Model A":22s} {"Model B":22s} {"t-stat":>8} {"p-value":>10} {"Significant?":>12}')
print('-' * 80)

ttest_rows = []
for m1, m2 in combinations(model_names, 2):
    t_stat, p_val = stats.ttest_rel(cv_results[m1], cv_results[m2])
    sig = 'YES 🟢' if p_val < 0.05 else 'NO 🛑'
    print(f'{m1:22s} {m2:22s} {t_stat:+8.4f} {p_val:10.4f} {sig:>12}')
    ttest_rows.append({'Model A': m1, 'Model B': m2,
                      't-stat': round(t_stat, 4),
                      'p-value': round(p_val, 4),
                      'Significant (p<0.05)': p_val < 0.05})

ttest_df = pd.DataFrame(ttest_rows)
ttest_df.to_csv('ttest_results.csv', index=False)
print('\n🟢 ttest_results.csv saved.')

```

📊 Paired t-test on 10-Fold CV F1 Scores (p < 0.05 = significant)

Model A	Model B	t-stat	p-value	Significant?
Logistic Regression	Decision Tree	-33.7403	0.0000	YES 🟢
Logistic Regression	Random Forest	-51.0765	0.0000	YES 🟢
Logistic Regression	KNN	-160.3439	0.0000	YES 🟢
Logistic Regression	XGBoost	-81.7026	0.0000	YES 🟢
Decision Tree	Random Forest	-31.0964	0.0000	YES 🟢
Decision Tree	KNN	-91.2871	0.0000	YES 🟢
Decision Tree	XGBoost	-32.2164	0.0000	YES 🟢
Random Forest	KNN	-42.2379	0.0000	YES 🟢
Random Forest	XGBoost	+7.2447	0.0000	YES 🟢

KNN

XGBoost

+134.3491

0.0000

YES



ttest_results.csv saved.

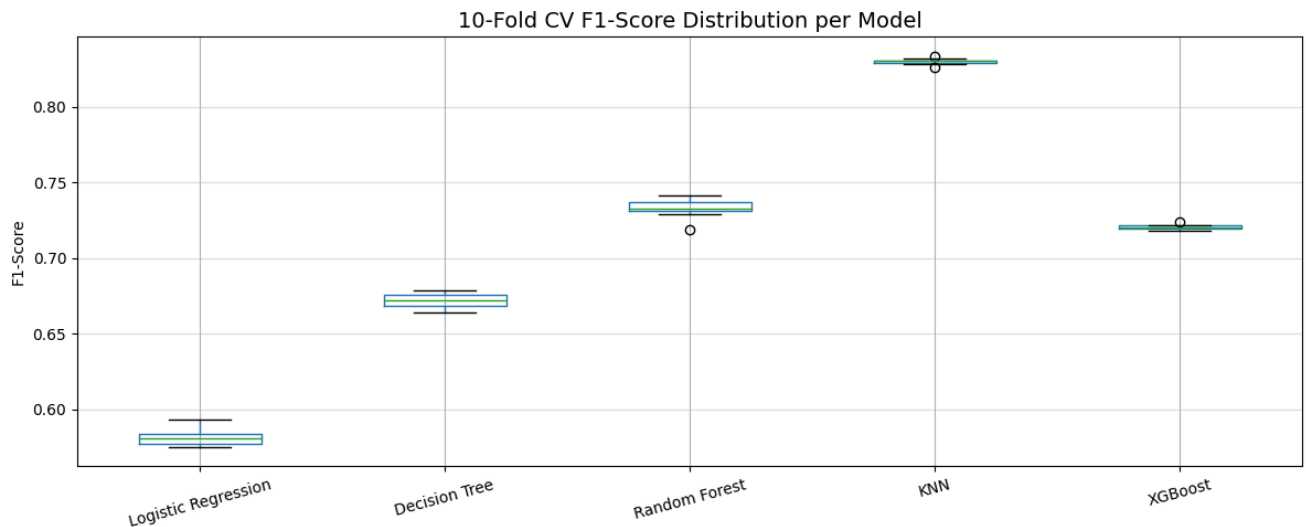
Cell 24 — CV F1-Score Distribution Boxplot

```

cv_df = pd.DataFrame(cv_results)

plt.figure(figsize=(12, 5))
cv_df.boxplot(column=list(cv_results.keys()), vert=True)
plt.title('10-Fold CV F1-Score Distribution per Model', fontsize=14)
plt.ylabel('F1-Score')
plt.xticks(rotation=15)
plt.grid(axis='y', alpha=0.4)
plt.tight_layout()
plt.savefig('cv_f1_boxplot.png', dpi=150, bbox_inches='tight')
plt.show()

```



Cell 25 — KNN Elbow Plot (Optimal K Selection)

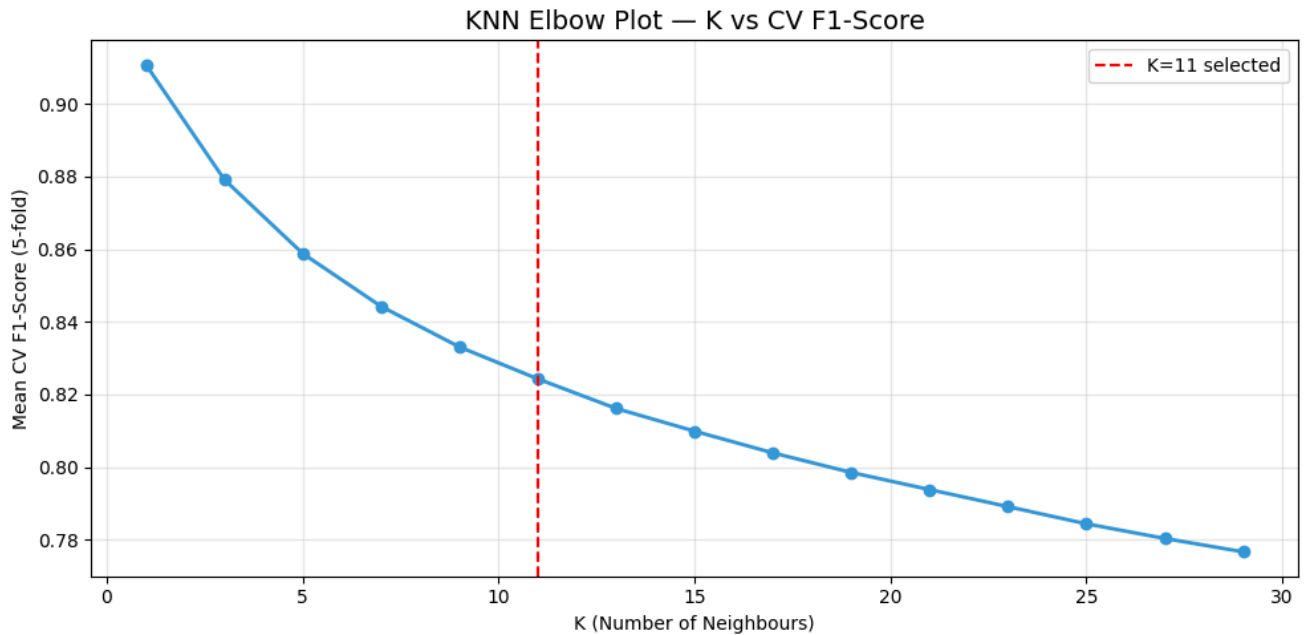
```

k_range = range(1, 31, 2)
k_f1_scores = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    score = cross_val_score(knn, X_train_sm, y_train_sm,
                            cv=5, scoring='f1', n_jobs=-1).mean()
    k_f1_scores.append(score)

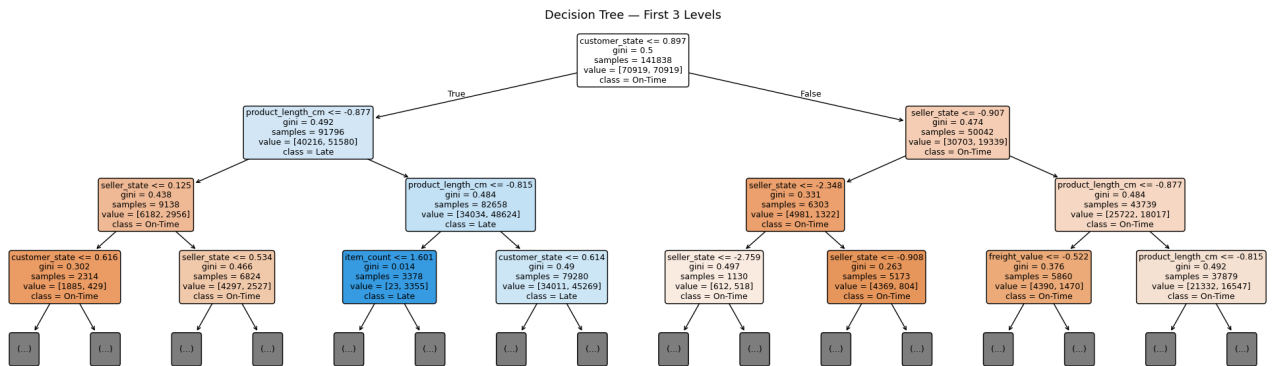
plt.figure(figsize=(10, 5))
plt.plot(list(k_range), k_f1_scores, marker='o', color='#3498db', lw=2)
plt.title('KNN Elbow Plot - K vs CV F1-Score', fontsize=14)
plt.xlabel('K (Number of Neighbours)')
plt.ylabel('Mean CV F1-Score (5-fold)')
plt.axvline(11, color='red', linestyle='--', label='K=11 selected')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('knn_elbow.png', dpi=150, bbox_inches='tight')
plt.show()

```



Cell 26 — Decision Tree Visualisation (First 3 Levels)

```
fig, ax = plt.subplots(figsize=(20, 6))
plot_tree(test_results['Decision Tree']['model'],
          feature_names=feature_cols,
          class_names=['On-Time', 'Late'],
          filled=True, rounded=True,
          max_depth=3, ax=ax, fontsize=9)
ax.set_title('Decision Tree — First 3 Levels', fontsize=13)
plt.tight_layout()
plt.savefig('decision_tree.png', dpi=150, bbox_inches='tight')
plt.show()
```



Cell 27 — Final Model Performance Comparison Bar Chart

```
metrics = ['F1-Score', 'ROC-AUC', 'Accuracy']
values = {m: [test_results[n][k] for k in ['f1', 'auc', 'accuracy']]
          for m, n in [('LR', 'Logistic Regression'), ('DT', 'Decision Tree'),
                      ('RF', 'Random Forest'), ('KNN', 'KNN'), ('XGB', 'XGBoost')]}

x = np.arange(len(metrics))
width = 0.15
fig, ax = plt.subplots(figsize=(13, 6))
colors = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12', '#9b59b6']

for i, (label, vals) in enumerate(values.items()):
    ax.bar(x + i * width, vals, width, label=label, color=colors[i])
```

```
ax.set_xticks(x + width * 2)
ax.set_xticklabels(metrics, fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Model Performance Comparison - All Metrics', fontsize=14)
ax.set_ylim(0, 1.1)
ax.legend(title='Model', fontsize=10)
ax.axhline(0.8, color='gray', linestyle='--', alpha=0.5, label='0.8 target')
plt.tight_layout()
plt.savefig('model_comparison_bar.png', dpi=150, bbox_inches='tight')
plt.show()
```

